

Axios in React

How Axios fetches data from a Spring Boot backend

Separate study notes - same style as the earlier PDF

- ◆ Why Axios instead of hardcoded JSON
- ◆ Reusable api.js configuration
- ◆ GET requests in useEffect
- ◆ POST, PUT and DELETE with Spring Boot
- ◆ Why state, router and forms still matter
- ◆ Error handling and common structure choices

React state stores what you see. Axios moves data between React and Spring Boot.

01 Why Axios belongs in this React app

Earlier, you learned to store data in **useState**, react to changes with **useEffect**, collect input values with controlled forms, and navigate with **React Router**. Axios fits into that same mental model: React still controls the UI, while Axios handles the HTTP work.

A hardcoded array is fine for learning the UI, but it has one big problem: the data lives only in the browser. Refresh the page and everything disappears. A **Spring Boot backend** gives the data a real home on the server.

The data flow

React component renders the page.
useEffect asks Axios to talk to Spring Boot.
Spring Boot reads or writes data in the database.
Axios returns JSON to React.
React stores that JSON in state and re-renders the UI.

That is why the code is structured so that the request happens in one place, and the UI just reads from state. React does not talk to the database directly. It only talks to the server through Axios.

02 Creating one reusable Axios instance

Instead of writing the full backend URL again and again, create a small Axios instance in one file. This keeps the code cleaner and makes future changes easy.

```
api.js
import axios from 'axios';

export default axios.create({
  baseURL: 'http://localhost:8080'
});
```

The important idea is that **baseURL** becomes the starting point for every request. If the Spring Boot backend runs on a different port, you change it here once, not in every component.

Why this file is written separately

- It keeps API details out of the component tree.
- The same api object can be imported anywhere.
- One update point for the backend URL.
- It matches the idea of reusable code that you already use with components.

03 Fetching posts with useEffect + axios.get

Fetching is a side effect, so it belongs inside **useEffect**. You do not want to fetch on every render - only when the component first loads.

In your app, **posts** starts as an empty array. After the request succeeds, the server response is copied into React state. That state then drives the rest of the page: search, display, edit and delete all work from the same source.

App.jsx

```
useEffect(() => {
  const fetchPosts = async () => {
    try {
      const response = await api.get('/posts');
      setPosts(response.data);
    } catch (err) {
      if (err.response) {
        console.log(err.response.data);
        console.log(err.response.status);
        console.log(err.response.headers);
      } else {
        console.log(`Error: ${err.message}`);
      }
    }
  }

  fetchPosts();
}, []);
```

Why each line is placed there

The async function sits inside useEffect because the effect itself cannot be marked async directly.

The empty dependency array [] means 'run once after the first render'.

response.data is the JSON body returned by Spring Boot.

setPosts(response.data) moves the backend data into React state so the UI can render it.

The error block is useful because Axios gives two different kinds of failure information. **err.response** means the server replied but with a non-2xx status code. **err.message** is for network or request problems where no valid response came back.

04 Search stays inside React state

The search feature is not Axios itself, but it shows how fetched data fits with the React patterns you already know. Once posts are in state, React can derive a filtered list from them without asking the server again.

Filtering

```
useEffect(() => {  
  const filteredResults = posts.filter((post) =>  
    ((post.body).toLowerCase()).includes(search.toLowerCase())  
    || ((post.title).toLowerCase()).includes(search.toLowerCase()));  
  
  setSearchResults(filteredResults.reverse());  
}, [posts, search]);
```

Why it is written this way

posts is the source data that came from Axios.

search is the live input from the user.

useEffect re-computes the results whenever either one changes.

setSearchResults stores the derived list so the UI can render it immediately.

This is a good example of React thinking: Axios fetches once, then React can transform that data many times locally.

05 POST request: creating a new blog post

When the user submits the form, React collects the input values from state, turns them into one object, and sends that object to Spring Boot.

This is exactly the kind of work Axios is good at: sending JSON bodies in a simple, readable way.

```
handleSubmit()  
  
const handleSubmit = async (e) => {  
  e.preventDefault();  
  const id = posts.length ? posts[posts.length - 1].id + 1 : 1;  
  const datetime = format(new Date(), 'MMMM dd, yyyy pp');  
  const newPost = { id, title: postTitle, datetime, body: postBody };  
  
  try {  
    const response = await api.post('/posts', newPost);  
    const allPosts = [...posts, response.data];  
    setPosts(allPosts);  
    setPostTitle('');  
    setPostBody('');  
    history.push('/');  
  } catch (err) {  
    console.log(`Error: ${err.message}`);  
  }  
};
```

Why these lines matter

`e.preventDefault()` stops the browser from reloading the page.

`newPost` is a plain JavaScript object, which is exactly what Axios will JSON-encode.

`api.post('/posts', newPost)` sends a POST request to Spring Boot.

`response.data` is usually the saved object returned by the backend.

`setPosts(allPosts)` makes the new post appear immediately in React.

`history.push('/')` sends the user back to the home page after success.

In Spring Boot, this maps naturally to a controller method that receives the JSON body with **@RequestBody** and saves it.

06 PUT request: editing an existing post

Editing looks similar to creating, but there is one extra idea: the form must first be filled with the current post values. That is why the edit component reads the **id** from the route and finds the matching post from state.

EditPost.jsx

```
const EditPost = ({ posts, handleEdit, editBody, setEditBody, editTitle, setEditTitle }) => {  
  const { id } = useParams();  
  const post = posts.find(post => (post.id).toString() === id);  
  
  useEffect(() => {  
    if (post) {  
      setEditTitle(post.title);  
      setEditBody(post.body);  
    }  
  }, [post, setEditTitle, setEditBody]);  
}
```

The effect here exists only to copy the selected post into the form state once the correct post has been found. That is why it depends on **post** and not on the input values themselves.

handleEdit()

```
const handleEdit = async (id) => {  
  const datetime = format(new Date(), 'MMMM dd, yyyy pp');  
  const updatedPost = { id, title: editTitle, datetime, body: editBody };  
  
  try {  
    const response = await api.put(`/posts/${id}`, updatedPost);  
    setPosts(posts.map(post => post.id === id ? { ...response.data } : post));  
    setEditTitle('');  
    setEditBody('');  
    history.push('/');  
  } catch (err) {  
    console.log(`Error: ${err.message}`);  
  }  
};
```

Why PUT is written this way

The id belongs in the URL because it identifies which record should change.

The updated content belongs in the request body.

setPosts(posts.map(...)) replaces only the matching item in local state.

history.push('/') makes the user land back on the list view after saving.

07 DELETE request: removing a post

Deleting is even simpler. Usually no body is needed; the id in the URL is enough. After the backend deletes the record, React filters it out of local state so the UI changes immediately.

`PostPage.jsx`

```
const handleDelete = async (id) => {
  try {
    await api.delete(`/posts/${id}`);
    const postsList = posts.filter(post => post.id !== id);
    setPosts(postsList);
    history.push('/');
  } catch (err) {
    console.log(`Error: ${err.message}`);
  }
};
```

Why filter instead of refetching

filter creates a new array without the deleted post.

This keeps the UI fast because React does not need to ask the server for everything again.

The backend remains the source of truth, but the client updates itself immediately.

The delete button in the post page is wired to this function. That is the same pattern you already used with button click events and prop passing.

08 How Axios fits with React Router

React Router is what makes the app feel like separate pages, even though the browser is still on one React application. Axios and React Router work together: Axios fetches or changes the data, while Router changes the view.

Where the router matters

Route / shows the list view.

Route /post shows the create form.

Route /edit/:id opens the edit page for one item.

Route /post/:id shows one post detail page.

history.push('/') moves the user after successful create, edit or delete.

App routes

```
<Route exact path="/">
  <Home posts={searchResults} />
</Route>

<Route exact path="/post">
  <NewPost handleSubmit={handleSubmit} ... />
</Route>

<Route path="/edit/:id">
  <EditPost posts={posts} handleEdit={handleEdit} ... />
</Route>

<Route path="/post/:id">
  <PostPage posts={posts} handleDelete={handleDelete} />
</Route>
```

The route decides which component is visible. The component decides which Axios function will be used. That separation keeps the app easy to reason about.

09 How this looks on the Spring Boot side

Since your backend is Spring Boot, the server usually exposes REST endpoints such as GET /posts, POST /posts, PUT /posts/{id} and DELETE /posts/{id}. Axios sends JSON to those routes, and Spring Boot turns that JSON into Java objects.

Typical backend responsibilities

GET reads data from the database and returns JSON.

POST receives JSON with @RequestBody and saves a new record.

PUT receives a path id plus JSON and updates the record.

DELETE uses @PathVariable to remove the matching record.

If React and Spring Boot run on different ports, CORS must allow the frontend origin. That is not an Axios problem; it is a browser security rule handled by the backend configuration.

10 The simple mental model to remember

Hardcoded JSON is good for learning the UI, but real apps need a server. Axios is the layer that swaps the fake local array for real backend data.

One-line summary for each part

useState stores the current UI data.

useEffect decides when data should be fetched.

Axios talks to Spring Boot.

Controlled inputs collect form values.

React Router moves between views after each action.

Once that model clicks, the whole CRUD flow becomes predictable: load data, show it, edit it, save it, delete it, and keep the React state in sync with the server.